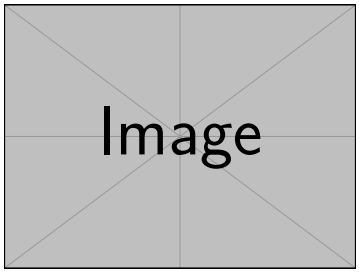
Fiche TP

Dashboard Statistique et Détermination Automatique des Types de Variables



Module : Statistique Descriptive

Thème : Représentations graphiques, paramètres et discrétisation

Outils : Python (Pandas, NumPy, Matplotlib)

Durée : 6 heures

Année universitaire 2025–2026

Table des matières

1 Objectifs pédagogiques **3**

2 Prérequis techniques **3**

3 Dataset d'exemple **3**

4 Détermination automatique des types de variables **4**
4.1 Principe 4

5 Tableaux statistiques **4**
5.1 Variable qualitative 5
5.2 Variable quantitative discrète 5
5.3 Variable quantitative continue : les trois règles de discrétisation 5

6 Représentations graphiques **7**
6.1 Tuyaux d'orgues (variable qualitative) 7
6.2 Diagramme circulaire 7
6.3 Diagramme en bâtons (quantitative discrète) 7
6.4 Courbe en escalier 7
6.5 Histogramme (quantitative continue) 8
6.6 Courbe cumulative (ogive) 8

7 Paramètres de position et de dispersion **9**
7.1 Définitions et formules 9
7.2 Calcul des paramètres sur une variable 9
7.3 Représentation graphique des paramètres 10

8 Détermination de Me, Q1 et Q3 à partir d'un histogramme **11**
8.1 Principe de l'interpolation linéaire 11
8.2 Exemple numérique sur le dataset 12
8.3 Tableau de calcul détaillé 13
8.4 Visualisation graphique de l'interpolation 13

9 Détermination de Me, Q1, Q2, Q3, D6 à partir de la courbe cumulative **14**
9.1 Principe de lecture graphique 14
9.2 Construction de la courbe cumulative 14
9.3 Tableau récapitulatif des quantiles 16
9.4 Comparaison lecture graphique vs calcul exact 16
9.5 Application complète : tous les quantiles sur toutes les variables 17
9.6 Interprétation des résultats 17

10 Diagramme de Tukey et détection des points aberrants **18**
10.1 Principe du diagramme de Tukey 18
10.2 Calcul des paramètres de Tukey 19
10.3 Détection automatique des points aberrants 19
10.4 Tracé du diagramme de Tukey 20
10.5 Comparaison avec une distribution normale 21
10.6 Interprétation du diagramme de Tukey 22
10.7 Application au dataset 22

11 Transformation discret → continu **22**

12 Opération inverse : graphique → tableau **23**
12.1 Reconstitution depuis un diagramme circulaire 23
12.2 Reconstitution depuis une courbe en escalier 23
12.3 Reconstitution depuis un histogramme 24

13 Dashboard de synthèse **24**

14 Synthèse des règles de discrétisation **26**

15 Synthèse des paramètres statistiques **26**

16 Exercices supplémentaires **27**

Bibliographie **27**

1. Objectifs pédagogiques

Objectifs pédagogiques

À l'issue de ce TP, l'étudiant sera capable de :

- Charger un dataset et **détecter automatiquement** le type de chaque variable :
  - qualitative nominale ou ordinale,
  - quantitative discrète ou continue.
- Générer un **tableau statistique** complet : effectifs, fréquences, fréquences cumulées.
- Tracer les représentations graphiques adaptées :
  - **tuyaux d'orgues** et **diagramme circulaire** (variables qualitatives),
  - **diagramme en bâtons** et **courbe en escalier** (variables quantitatives discrètes),
  - **histogramme** et **courbe cumulative** (variables quantitatives continues).
- Calculer et représenter graphiquement les **paramètres de position** (moyenne, médiane, mode, quartiles) et de **dispersion** (étendue, IQR, variance, écart-type, CV).
- Déterminer **Me, Q1, Q3** à partir d'un **histogramme** par interpolation linéaire.
- Déterminer **Me, Q1, Q2, Q3, D6** à partir de la **courbe cumulative**.
- Construire un **diagramme de Tukey** (boîte à moustaches) et **détecter les points aberrants** selon le critère de Tukey.
- Transformer des données quantitatives **discrètes en continues** via les règles de **Sturges**, **Yule** et **Freedman–Diaconis**.
- Réaliser l'**opération inverse** : reconstituer à partir d'un graphique le nom de la variable, la population, le type et le tableau de données.

2. Prérequis techniques

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import warnings
warnings.filterwarnings('ignore')
```

Listing 1 – Imports nécessaires

3. Dataset d'exemple

```python
data = {
    'ID': range(1, 51),
    'Genre': ['M', 'F', 'M', 'F', 'M', 'F', 'M', 'F', 'M', 'F'] * 5,
    'Niveau_Etude': ['Licence', 'Master', 'Doctorat', 'Licence', 'Master',
                     'Licence', 'Master', 'Doctorat', 'Licence', 'Master']
                         * 5,
    'Age': [20, 22, 25, 21, 23, 26, 24, 27, 20, 22] * 5,
    'Note_Stats': [12.5, 15.0, 9.5, 17.0, 11.0, 14.5, 13.0, 16.5, 10.5,
       18.0] * 5,
    'Heures_Travail': [10, 15, 8, 20, 12, 18, 14, 16, 9, 22] * 5
}
df = pd.DataFrame(data)
```

```python

# Alternative : chargement depuis un fichier CSV
# df = pd.read_csv('votre_fichier.csv')

print("Aperçu du dataset :")
print(df.head())
print("\nDimensions :", df.shape)
```

Listing 2 – Chargement du dataset

4. Détermination automatique des types de variables

4.1 Principe

Pandas ne classifie pas automatiquement les variables en qualitatives/quantitatives. Nous implémentons une fonction de détection basée sur le `dtype` et le nombre de valeurs uniques.

```python
def detecter_type_variable(serie):
    """
    Detecte automatiquement le type statistique d'une variable.
    Retourne :
    - 'qualitative_nominale' : categories sans ordre
    - 'qualitative_ordinale' : categories avec ordre possible
    - 'quantitative_discrete' : valeurs numeriques finies
    - 'quantitative_continue' : valeurs numeriques mesurables
    """
    dtype = serie.dtype
    n_unique = serie.nunique()

    if dtype in ['object', 'category', 'bool'] or n_unique < 10:
        categories = serie.dropna().unique()
        ordre_possible = any(mot in str(categories).lower()
                             for mot in ['niveau', 'faible', 'moyen', 'eleve
                                 '])
        return 'qualitative_ordinale' if ordre_possible else '
            qualitative_nominale'

    elif np.issubdtype(dtype, np.number):
        if np.all(serie.dropna() == serie.dropna().astype(int)):
            return 'quantitative_discrete' if n_unique <= 20 else '
                quantitative_continue'
        else:
            return 'quantitative_continue'

    return 'inconnu'

# Application
types_variables = {}
for col in df.columns:
    type_var = detecter_type_variable(df[col])
    types_variables[col] = type_var
    print(f"{col:20s} -> {type_var}")
```

Listing 3 – Fonction de détection automatique

5. Tableaux statistiques

5.1 Variable qualitative

```python
def tableau_qualitatif(df, colonne):
    effectifs = df[colonne].value_counts().sort_index()
    frequences = df[colonne].value_counts(normalize=True).sort_index()

    tableau = pd.DataFrame({
        'Modalite': effectifs.index,
        'Effectif (n_i)': effectifs.values,
        'Frequence (f_i)': frequences.values,
        'Frequence (%)': (frequences.values * 100).round(2)
    })
    tableau['Frequence cumulee'] = tableau['Frequence (f_i)'].cumsum()
    return tableau

print("Tableau statistique - Genre :")
print(tableau_qualitatif(df, 'Genre'))
```

Listing 4 – Tableau statistique qualitatif

5.2 Variable quantitative discrète

```python
def tableau_quantitatif_discret(df, colonne):
    effectifs = df[colonne].value_counts().sort_index()
    frequences = df[colonne].value_counts(normalize=True).sort_index()

    tableau = pd.DataFrame({
        'Valeur (x_i)': effectifs.index,
        'Effectif (n_i)': effectifs.values,
        'Frequence (f_i)': frequences.values,
        'Frequence cumulee': frequences.values.cumsum()
    })
    return tableau

print(tableau_quantitatif_discret(df, 'Age'))
```

Listing 5 – Tableau statistique quantitatif discret

5.3 Variable quantitative continue : les trois règles de discrétisation

Règle de Sturges

$$k = 1 + 3{,}322 \cdot \log_{10}(n)$$

Adaptée aux distributions proches de la loi normale, pour $n < 200$.

Règle de Yule

$$k = 2{,}5 \cdot n^{1/4}$$

Utilisable pour des distributions quelconques, $n < 1000$.

Règle de Freedman–Diaconis

$$h = \frac{2 \cdot \text{IQR}}{n^{1/3}}, \qquad k = \left\lceil \frac{\max - \min}{h} \right\rceil$$

Robuste aux valeurs aberrantes (outliers).

```python
def creer_classes_sturges(serie):
    n = len(serie)
    nb_classes = int(np.ceil(1 + 3.322 * np.log10(n)))
    etendue = serie.max() - serie.min()
    amplitude = etendue / nb_classes
    bornes = [serie.min() + i * amplitude for i in range(nb_classes + 1)]
    bornes[-1] = serie.max()
    return bornes, nb_classes

def creer_classes_yule(serie):
    n = len(serie)
    nb_classes = int(np.ceil(2.5 * n ** 0.25))
    etendue = serie.max() - serie.min()
    amplitude = etendue / nb_classes
    bornes = [serie.min() + i * amplitude for i in range(nb_classes + 1)]
    bornes[-1] = serie.max()
    return bornes, nb_classes

def creer_classes_freedman_diaconis(serie):
    n = len(serie)
    q25, q75 = np.percentile(serie, [25, 75])
    iqr = q75 - q25
    h = 2 * iqr / (n ** (1/3))
    if h == 0:
        h = (serie.max() - serie.min()) / 10
    etendue = serie.max() - serie.min()
    nb_classes = int(np.ceil(etendue / h))
    bornes = [serie.min() + i * h for i in range(nb_classes + 1)]
    bornes[-1] = serie.max()
    return bornes, nb_classes

def tableau_quantitatif_continu(df, colonne, methode='sturges'):
    serie = df[colonne].dropna()
    if methode == 'sturges':
        bornes, nb_classes = creer_classes_sturges(serie)
    elif methode == 'yule':
        bornes, nb_classes = creer_classes_yule(serie)
    elif methode == 'fd':
        bornes, nb_classes = creer_classes_freedman_diaconis(serie)

    effectifs, _ = np.histogram(serie, bins=bornes)
    frequences = effectifs / len(serie)
    frequences_cumulees = np.cumsum(frequences)
    labels = [f"[{bornes[i]:.2f} ; {bornes[i+1]:.2f}]"
              for i in range(len(bornes)-1)]

    return pd.DataFrame({
        'Classe': labels,
        'Effectif (n_i)': effectifs,
        'Frequence (f_i)': frequences.round(4),
        'Frequence cumulee': frequences_cumulees.round(4)
    })
```

Listing 6 – Création des classes selon les trois règles

6. Représentations graphiques

6.1 Tuyaux d'orgues (variable qualitative)

```
def tracer_tuyaux_orgues(df, colonne, couleur='steelblue'):
    effectifs = df[colonne].value_counts().sort_index()
    fig, ax = plt.subplots(figsize=(8, 5))
    ax.bar(effectifs.index, effectifs.values, color=couleur,
           edgecolor='black', width=0.6)
    ax.set_xlabel(colonne)
    ax.set_ylabel("Effectif")
    ax.set_title(f"Diagramme en tuyaux d'orgues - {colonne}")
    ax.grid(axis='y', alpha=0.3)
    plt.tight_layout(); plt.show()

tracer_tuyaux_orgues(df, 'Niveau_Etude', couleur='coral')
```

6.2 Diagramme circulaire

```
def tracer_diagramme_circulaire(df, colonne):
    effectifs = df[colonne].value_counts()
    fig, ax = plt.subplots(figsize=(7, 7))
    wedges, texts, autotexts = ax.pie(
        effectifs.values, labels=effectifs.index, autopct='%1.1f%%',
        colors=sns.color_palette("pastel"), startangle=90,
        wedgeprops={'edgecolor': 'white', 'linewidth': 2})
    ax.set_title(f"Diagramme circulaire - {colonne}", fontsize=14)
    plt.tight_layout(); plt.show()

tracer_diagramme_circulaire(df, 'Genre')
```

6.3 Diagramme en bâtons (quantitative discrète)

```
def tracer_diagramme_batons(df, colonne, couleur='mediumseagreen'):
    effectifs = df[colonne].value_counts().sort_index()
    fig, ax = plt.subplots(figsize=(8, 5))
    ax.bar(effectifs.index, effectifs.values, color=couleur,
           edgecolor='black', width=0.4)
    ax.set_xlabel(colonne); ax.set_ylabel("Effectif")
    ax.set_title(f"Diagramme en bâtons - {colonne}")
    ax.grid(axis='y', alpha=0.3)
    plt.tight_layout(); plt.show()

tracer_diagramme_batons(df, 'Age')
```

6.4 Courbe en escalier

```
def tracer_courbe_escalier(df, colonne, type_var='discrete'):
    if type_var == 'discrete':
        effectifs = df[colonne].value_counts().sort_index()
```

```
        f_cum = effectifs.cumsum() / effectifs.sum()
        x = effectifs.index
    else:
        bornes, _ = creer_classes_sturges(df[colonne])
        effectifs, _ = np.histogram(df[colonne], bins=bornes)
        f_cum = np.cumsum(effectifs) / len(df[colonne])
        x = bornes[1:]

    fig, ax = plt.subplots(figsize=(8, 5))
    ax.step(x, f_cum, where='post', color='darkorange', linewidth=2.5)
    ax.scatter(x, f_cum, color='red', zorder=5)
    ax.set_xlabel(colonne); ax.set_ylabel("Frequence cumulee")
    ax.set_title(f"Courbe en escalier - {colonne}")
    ax.grid(alpha=0.3); ax.set_ylim(0, 1.05)
    plt.tight_layout(); plt.show()

tracer_courbe_escalier(df, 'Age', type_var='discrete')
```

6.5 Histogramme (quantitative continue)

```
def tracer_histogramme(df, colonne, methode='sturges', couleur='
    mediumpurple'):
    serie = df[colonne].dropna()
    if methode == 'sturges':
        bornes, _ = creer_classes_sturges(serie); titre = "Sturges"
    elif methode == 'yule':
        bornes, _ = creer_classes_yule(serie); titre = "Yule"
    elif methode == 'fd':
        bornes, _ = creer_classes_freedman_diaconis(serie)
        titre = "Freedman-Diaconis"

    fig, ax = plt.subplots(figsize=(8, 5))
    ax.hist(serie, bins=bornes, color=couleur, edgecolor='black', alpha
        =0.8)
    ax.set_xlabel(colonne); ax.set_ylabel("Effectif")
    ax.set_title(f"Histogramme - {colonne} (regle de {titre})")
    ax.grid(axis='y', alpha=0.3)
    plt.tight_layout(); plt.show()

tracer_histogramme(df, 'Note_Stats', 'sturges')
tracer_histogramme(df, 'Note_Stats', 'yule', couleur='lightcoral')
tracer_histogramme(df, 'Note_Stats', 'fd', couleur='lightblue')
```

6.6 Courbe cumulative (ogive)

```
def tracer_courbe_cumulative(df, colonne, methode='sturges'):
    serie = df[colonne].dropna()
    if methode == 'sturges':
        bornes, _ = creer_classes_sturges(serie)
    elif methode == 'yule':
        bornes, _ = creer_classes_yule(serie)
    elif methode == 'fd':
        bornes, _ = creer_classes_freedman_diaconis(serie)

    effectifs, _ = np.histogram(serie, bins=bornes)
    f_cum = np.cumsum(effectifs) / len(serie)

```

```
    fig, ax = plt.subplots(figsize=(8, 5))
    ax.plot(bornes[1:], f_cum, marker='o', color='teal', linewidth=2)
    ax.fill_between(bornes[1:], f_cum, alpha=0.2,
                    color='teal', step='post')
    ax.set_xlabel(colonne); ax.set_ylabel("Frequence cumulee")
    ax.set_title(f"Courbe cumulative (ogive) - {colonne}")
    ax.grid(alpha=0.3); ax.set_ylim(0, 1.05)
    plt.tight_layout(); plt.show()

tracer_courbe_cumulative(df, 'Note_Stats', 'sturges')
```

7. Paramètres de position et de dispersion

7.1 Définitions et formules

Catégorie	Paramètre	Formule
Position	Moyenne $\bar{x}$	$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$
	Médiane Me	Valeur qui partage la série en 2 parties égales
	Mode Mo	Valeur de plus grand effectif
	Quartiles Q_1, Q_2, Q_3	Valeurs qui partagent en 4 parties égales
Dispersion	Étendue	$E = x_{max} - x_{min}$
	Écart interquartile	$IQR = Q_3 - Q_1$
	Variance s^2	$s^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$
	Écart-type s	$s = \sqrt{s^2}$
Forme	Coefficient de variation	$CV = \frac{s}{\bar{x}}$
	Coefficient d'asymétrie	$\gamma_1 = \frac{1}{n}\sum\left(\frac{x_i - \bar{x}}{s}\right)^3$

7.2 Calcul des paramètres sur une variable

```
def parametres_position(serie):
    """Calcule les parametres de position."""
    serie = serie.dropna()
    return {
        'Effectif (n)': len(serie),
        'Moyenne': serie.mean(),
        'Mediane (Q2)': serie.median(),
        'Mode': serie.mode().iloc[0] if len(serie.mode()) > 0 else np.nan,
        'Minimum': serie.min(),
        'Maximum': serie.max(),
        'Q1': np.percentile(serie, 25),
        'Q3': np.percentile(serie, 75)
    }

```

```python
def parametres_dispersion(serie):
    """Calcule les parametres de dispersion."""
    serie = serie.dropna()
    q1, q3 = np.percentile(serie, [25, 75])
    return {
        'Etendue': serie.max() - serie.min(),
        'IQR (Q3-Q1)': q3 - q1,
        'Variance': serie.var(ddof=1),
        'Ecart-type': serie.std(ddof=1),
        'CV (%)': (serie.std(ddof=1) / serie.mean() * 100).round(2),
        'Asymetrie': serie.skew(),
        'Aplatissement': serie.kurtosis()
    }

# Application
print("=" * 60)
print("PARAMETRES DE POSITION - Note_Stats")
print("=" * 60)
for k, v in parametres_position(df['Note_Stats']).items():
    print(f"  {k:20s} = {v:.4f}")

print("\n" + "=" * 60)
print("PARAMETRES DE DISPERSION - Note_Stats")
print("=" * 60)
for k, v in parametres_dispersion(df['Note_Stats']).items():
    print(f"  {k:20s} = {v:.4f}")
```

Listing 7 – Calcul complet des parametres de position et dispersion

7.3 Représentation graphique des paramètres

```python
def visualiser_parametres_position(df, colonne):
    """Affiche les parametres de position sur l'histogramme."""
    serie = df[colonne].dropna()
    p = parametres_position(serie)

    fig, ax = plt.subplots(figsize=(10, 5))
    ax.hist(serie, bins=15, color='lightblue', edgecolor='black', alpha
        =0.7)

    # Lignes verticales pour les parametres de position
    ax.axvline(p['Moyenne'], color='red', linestyle='-', linewidth=2,
               label=f"Moyenne = {p['Moyenne']:.2f}")
    ax.axvline(p['Mediane (Q2)'], color='green', linestyle='--', linewidth
        =2,
               label=f"Mediane = {p['Mediane (Q2)']:.2f}")
    ax.axvline(p['Q1'], color='orange', linestyle=':', linewidth=2,
               label=f"Q1 = {p['Q1']:.2f}")
    ax.axvline(p['Q3'], color='purple', linestyle=':', linewidth=2,
               label=f"Q3 = {p['Q3']:.2f}")

    # Moyenne avec marqueur
    ax.scatter([p['Moyenne']], [0], color='red', marker='D', s=100,
               zorder=5, clip_on=False)

    ax.set_xlabel(colonne)
    ax.set_ylabel("Effectif")
    ax.set_title(f"Parametres de position - {colonne}")
    ax.legend(loc='upper right')
```

```
    ax.grid(alpha=0.3)
    plt.tight_layout(); plt.show()

visualiser_parametres_position(df, 'Note_Stats')
visualiser_parametres_position(df, 'Heures_Travail')
```

Listing 8 – Visualisation graphique des parametres de position

```
def visualiser_dispersion(df, colonne):
    """Affiche l'etendue, l'IQR et l'ecart-type."""
    serie = df[colonne].dropna()
    p_pos = parametres_position(serie)
    p_dis = parametres_dispersion(serie)

    fig, ax = plt.subplots(figsize=(10, 4))
    ax.hist(serie, bins=15, color='lightgreen', edgecolor='black', alpha
        =0.6)

    # Etendue
    ax.annotate('', xy=(p_pos['Minimum'], 0), xytext=(p_pos['Maximum'], 0)
        ,
                arrowprops=dict(arrowstyle='<->', color='red', lw=2))
    ax.text((p_pos['Minimum'] + p_pos['Maximum'])/2, 1,
            f"Etendue = {p_dis['Etendue']:.2f}",
            ha='center', color='red', fontsize=10)

    # IQR
    y_iqr = 3
    ax.annotate('', xy=(p_pos['Q1'], y_iqr), xytext=(p_pos['Q3'], y_iqr),
                arrowprops=dict(arrowstyle='<->', color='blue', lw=2))
    ax.text((p_pos['Q1'] + p_pos['Q3'])/2, y_iqr + 0.5,
            f"IQR = {p_dis['IQR (Q3-Q1)']:.2f}",
            ha='center', color='blue', fontsize=10)

    # Ecart-type
    ax.axvline(p_pos['Moyenne'] - p_dis['Ecart-type'], color='purple',
               linestyle='--', linewidth=1.5, alpha=0.7)
    ax.axvline(p_pos['Moyenne'] + p_dis['Ecart-type'], color='purple',
               linestyle='--', linewidth=1.5, alpha=0.7,
               label=f"$\\bar{{x}} \\pm s$ ({p_dis['Ecart-type']:.2f})")
    ax.axvline(p_pos['Moyenne'], color='red', linewidth=2, label='Moyenne'
        )

    ax.set_xlabel(colonne)
    ax.set_ylabel("Effectif")
    ax.set_title(f"Parametres de dispersion - {colonne}")
    ax.legend(loc='upper right')
    plt.tight_layout(); plt.show()

visualiser_dispersion(df, 'Note_Stats')
```

Listing 9 – Visualisation des parametres de dispersion

8. Détermination de Me, Q1 et Q3 à partir d'un histogramme

8.1 Principe de l'interpolation linéaire

Lorsque les données sont regroupées en classes, on ne connaît pas les valeurs exactes. On utilise l'**interpolation linéaire** à l'intérieur de la classe contenant le quantile recherché.

Formule générale d'interpolation

Pour un quantile Q_p (où $p \in \{0{,}25\,;\,0{,}50\,;\,0{,}75\}$) :

$$Q_p = L_i + \frac{p \cdot n - N_{i-1}}{n_i} \times a_i$$

où :

- L_i : borne inférieure de la classe contenant Q_p,
- N_{i-1} : effectif cumulé **avant** la classe i,
- n_i : effectif de la classe i,
- a_i : amplitude de la classe i,
- n : effectif total.

8.2 Exemple numérique sur le dataset

Considérons la variable `Note_Stats` discrétisée en classes (règle de Sturges).

```
def quantiles_par_interpolation(serie, methode='sturges'):
    """
    Determine Me, Q1, Q3 par interpolation lineaire
    a partir d'un histogramme (donnees groupees).
    """
    serie = serie.dropna()
    n = len(serie)

    # Construction des classes
    if methode == 'sturges':
        bornes, k = creer_classes_sturges(serie)
    elif methode == 'yule':
        bornes, k = creer_classes_yule(serie)
    elif methode == 'fd':
        bornes, k = creer_classes_freedman_diaconis(serie)

    effectifs, _ = np.histogram(serie, bins=bornes)
    effectifs_cum = np.cumsum(effectifs)
    amplitudes = np.diff(bornes)

    def interpoler(p):
        cible = p * n
        # Trouver la classe i telle que N_{i-1} < cible <= N_i
        i = np.searchsorted(effectifs_cum, cible, side='left')
        if i == 0:
            N_avant = 0
        else:
            N_avant = effectifs_cum[i-1]
        L_i = bornes[i]
        n_i = effectifs[i]
        a_i = amplitudes[i]
        return L_i + ((cible - N_avant) / n_i) * a_i

    return {
        'Q1': interpoler(0.25),
        'Me (Q2)': interpoler(0.50),
        'Q3': interpoler(0.75),
        'IQR (Q3-Q1)': interpoler(0.75) - interpoler(0.25),
        'bornes': bornes,
        'effectifs': effectifs,
```

```
        'effectifs_cum': effectifs_cum
    }

# Application
resultat = quantiles_par_interpolation(df['Note_Stats'], 'sturges')

print("=" * 60)
print("QUANTILES PAR INTERPOLATION - Note_Stats (Sturges)")
print("=" * 60)
print(f"  Q1         = {resultat['Q1']:.4f}")
print(f"  Me (Q2)    = {resultat['Me (Q2)']:.4f}")
print(f"  Q3         = {resultat['Q3']:.4f}")
print(f"  IQR        = {resultat['IQR (Q3-Q1)']:.4f}")

# Comparaison avec les valeurs exactes
print("\n--- Comparaison avec les valeurs exactes (donnees brutes) ---")
print(f"  Q1 exact   = {np.percentile(df['Note_Stats'], 25):.4f}")
print(f"  Me exact   = {np.percentile(df['Note_Stats'], 50):.4f}")
print(f"  Q3 exact   = {np.percentile(df['Note_Stats'], 75):.4f}")
```

Listing 10 – Construction des classes et determination des quantiles

8.3 Tableau de calcul détaillé

boxbg **Classe**	**Effectif** n_i	**Eff. cumulé** N_i	**Amplitude** a_i	**Position**
$[L_1\,;\,L_2[$	n_1	N_1	a_1	
$\vdots$	$\vdots$	$\vdots$	$\vdots$	
$[L_i\,;\,L_{i+1}[$	n_i	N_i	a_i	$\leftarrow$ classe de Q_p

8.4 Visualisation graphique de l'interpolation

```
def tracer_histogramme_quantiles(df, colonne, methode='sturges'):
    """Trace l'histogramme avec Me, Q1, Q3 interpoles."""
    serie = df[colonne].dropna()
    res = quantiles_par_interpolation(serie, methode)

    fig, ax = plt.subplots(figsize=(10, 5))
    ax.hist(serie, bins=res['bornes'], color='lightblue',
            edgecolor='black', alpha=0.7)

    # Lignes verticales
    ax.axvline(res['Q1'], color='orange', linestyle='--', linewidth=2,
               label=f"Q1 = {res['Q1']:.2f}")
    ax.axvline(res['Me (Q2)'], color='red', linestyle='-', linewidth=2,
               label=f"Me = {res['Me (Q2)']:.2f}")
    ax.axvline(res['Q3'], color='purple', linestyle='--', linewidth=2,
               label=f"Q3 = {res['Q3']:.2f}")

    # Zone IQR
    ax.axvspan(res['Q1'], res['Q3'], alpha=0.15, color='blue',
               label=f"IQR = {res['IQR (Q3-Q1)']:.2f}")

    ax.set_xlabel(colonne); ax.set_ylabel("Effectif")
    ax.set_title(f"Me, Q1, Q3 par interpolation - {colonne}")
    ax.legend(loc='upper right')
    ax.grid(alpha=0.3)
```

```
    plt.tight_layout(); plt.show()

tracer_histogramme_quantiles(df, 'Note_Stats', 'sturges')
```

Listing 11 – Histogramme avec quantiles interpoles

9. Détermination de Me, Q1, Q2, Q3, D6 à partir de la courbe cumulative

9.1 Principe de lecture graphique

La **courbe cumulative** (ogive) représente les fréquences cumulées croissantes en fonction des valeurs (ou bornes de classes). Pour déterminer un quantile Q_p :

1. On trace une **horizontale** à la hauteur p (ex. : 0,25 pour Q_1).
2. Cette horizontale coupe la courbe cumulative en un point.
3. On projette ce point sur l'**axe des abscisses** pour lire Q_p.

Correspondances quantiles / fréquences cumulées

Quantile	Fréquence cumulée cible
Q_1	0,25
$Q_2 = Me$	0,50
D_6 (6e décile)	0,60
Q_3	0,75

Remarque

Les **déciles** $D_1, D_2, \ldots, D_9$ partagent la série en 10 parties égales. Le k-ième décile D_k correspond à la fréquence cumulée $k/10$.

9.2 Construction de la courbe cumulative

```
def courbe_cumulative_quantiles(serie, methode='sturges'):
    """
    Construit la courbe cumulative et determine graphiquement
    Me, Q1, Q2, Q3, D6 par interpolation lineaire.
    """
    serie = serie.dropna()
    n = len(serie)

    if methode == 'sturges':
        bornes, k = creer_classes_sturges(serie)
    elif methode == 'yule':
        bornes, k = creer_classes_yule(serie)
    elif methode == 'fd':
        bornes, k = creer_classes_freedman_diaconis(serie)

    effectifs, _ = np.histogram(serie, bins=bornes)
    f_cum = np.cumsum(effectifs) / n
    amplitudes = np.diff(bornes)

    def interpoler(p):
```

```
        # Trouver la classe contenant la frequence cumulee p
        i = np.searchsorted(f_cum, p, side='left')
        if i == 0:
            F_avant = 0
        else:
            F_avant = f_cum[i-1]
        L_i = bornes[i]
        f_i = effectifs[i] / n
        a_i = amplitudes[i]
        return L_i + ((p - F_avant) / f_i) * a_i

    quantiles = {
        'Q1': interpoler(0.25),
        'Q2 (Me)': interpoler(0.50),
        'D6': interpoler(0.60),
        'Q3': interpoler(0.75)
    }

    return bornes, f_cum, quantiles

def tracer_courbe_cumulative_quantiles(df, colonne, methode='sturges'):
    """Trace la courbe cumulative avec les quantiles annotes."""
    serie = df[colonne].dropna()
    bornes, f_cum, quantiles = courbe_cumulative_quantiles(serie, methode)

    fig, ax = plt.subplots(figsize=(10, 6))

    # Courbe cumulative
    ax.plot(bornes[1:], f_cum, marker='o', color='teal',
            linewidth=2, label='Courbe cumulative')
    ax.fill_between(bornes[1:], f_cum, alpha=0.15,
                    color='teal', step='post')

    # Lignes horizontales pour chaque quantile
    couleurs = {'Q1': 'orange', 'Q2 (Me)': 'red',
                'D6': 'magenta', 'Q3': 'purple'}
    for nom, val in quantiles.items():
        p = {'Q1': 0.25, 'Q2 (Me)': 0.50,
             'D6': 0.60, 'Q3': 0.75}[nom]
        ax.axhline(p, color=couleurs[nom], linestyle='--',
                   linewidth=1.5, alpha=0.7)
        ax.axvline(val, color=couleurs[nom], linestyle=':',
                   linewidth=1.5, alpha=0.7)
        ax.scatter([val], [p], color=couleurs[nom], s=80, zorder=5)
        ax.text(val + 0.3, p + 0.02, f"{nom} = {val:.2f}",
                color=couleurs[nom], fontsize=9, fontweight='bold')

    ax.set_xlabel(colonne)
    ax.set_ylabel("Frequence cumulee")
    ax.set_title(f"Courbe cumulative - Determination de Me, Q1, Q2, Q3, D6
        ")
    ax.set_ylim(0, 1.05)
    ax.grid(alpha=0.3)
    ax.legend(loc='lower right')
    plt.tight_layout(); plt.show()

    return quantiles

# Application
```

```
quantiles = tracer_courbe_cumulative_quantiles(df, 'Note_Stats', 'sturges')
print("\nQuantiles determines graphiquement :")
for k, v in quantiles.items():
    print(f"  {k:10s} = {v:.4f}")
```

Listing 12 – Construction de la courbe cumulative et lecture des quantiles

9.3 Tableau récapitulatif des quantiles

boxbg **Quantile**	**Fréquence cumulée**	**Valeur lue**	**Interprétation**
Q_1	0,25	...	25% des individus ont une valeur $\leq Q_1$
$Q_2 = Me$	0,50	...	50% des individus ont une valeur $\leq Me$
D_6	0,60	...	60% des individus ont une valeur $\leq D_6$
Q_3	0,75	...	75% des individus ont une valeur $\leq Q_3$

9.4 Comparaison lecture graphique vs calcul exact

```
def comparer_quantiles(df, colonne, methode='sturges'):
    """Compare les quantiles obtenus par differentes methodes."""
    serie = df[colonne].dropna()

    # Methode 1 : donnees brutes (exacte)
    q_exact = {
        'Q1': np.percentile(serie, 25),
        'Q2 (Me)': np.percentile(serie, 50),
        'D6': np.percentile(serie, 60),
        'Q3': np.percentile(serie, 75)
    }

    # Methode 2 : interpolation sur histogramme
    _, _, q_interp = courbe_cumulative_quantiles(serie, methode)

    # Tableau comparatif
    comparaison = pd.DataFrame({
        'Quantile': list(q_exact.keys()),
        'Exact (donnees brutes)': [round(v, 4) for v in q_exact.values()],
        f'Interpole ({methode})': [round(q_interp[k], 4) for k in q_exact.
            keys()],
    })
    comparaison['Ecart'] = (
        comparaison[f'Interpole ({methode})']
        - comparaison['Exact (donnees brutes)']
    ).round(4)

    print(f"\nComparaison des quantiles - {colonne} ({methode})")
    print("=" * 70)
    print(comparaison.to_string(index=False))
    return comparaison

comparer_quantiles(df, 'Note_Stats', 'sturges')
comparer_quantiles(df, 'Note_Stats', 'yule')
comparer_quantiles(df, 'Note_Stats', 'fd')
```

Listing 13 – Comparaison des methodes

Remarque

Plus le nombre de classes est élevé, plus l'interpolation est précise. La méthode de **Freedman–Diaconis**, adaptée à la forme de la distribution, donne généralement la meilleure approximation.

9.5 Application complète : tous les quantiles sur toutes les variables

```python
def analyse_quantiles_complete(df):
    """Analyse complete des quantiles pour toutes les variables
       quantitatives."""
    print("=" * 70)
    print("ANALYSE DES QUANTILES - ME, Q1, Q2, Q3, D6")
    print("=" * 70)

    for col in df.columns:
        type_var = detecter_type_variable(df[col])
        if type_var in ['quantitative_discrete', 'quantitative_continue']:
            print(f"\n{'='*70}\nVARIABLE : {col} ({type_var})\n{'='*70}")

            serie = df[col].dropna()
            n = len(serie)

            # Calculs exacts
            q1 = np.percentile(serie, 25)
            me = np.percentile(serie, 50)
            d6 = np.percentile(serie, 60)
            q3 = np.percentile(serie, 75)
            iqr = q3 - q1

            print(f"  Effectif n      = {n}")
            print(f"  Q1              = {q1:.4f}")
            print(f"  Me (Q2)         = {me:.4f}")
            print(f"  D6              = {d6:.4f}")
            print(f"  Q3              = {q3:.4f}")
            print(f"  IQR = Q3 - Q1   = {iqr:.4f}")

            if type_var == 'quantitative_continue':
                tracer_courbe_cumulative_quantiles(df, col, 'sturges')

analyse_quantiles_complete(df)
```

Listing 14 – Application complete

9.6 Interprétation des résultats

Observation	Interprétation
$Me \approx \bar{x}$	Distribution symétrique
$Me < \bar{x}$	Distribution étalée à droite (asymétrie positive)
$Me > \bar{x}$	Distribution étalée à gauche (asymétrie négative)
IQR petit	Données concentrées autour de la médiane
IQR grand	Forte dispersion des 50% centraux
$D_6 - Me$ grand	Forte dispersion dans la partie supérieure

```python
def interpreter_distribution(df, colonne):
    """Interpretation automatique de la distribution."""
    serie = df[colonne].dropna()
    moyenne = serie.mean()
    med = serie.median()
    q1, q3 = np.percentile(serie, [25, 75])
    iqr = q3 - q1
    d6 = np.percentile(serie, 60)

    print(f"\n--- Interpretation de {colonne} ---")
    print(f"  Moyenne = {moyenne:.4f} | Mediane = {med:.4f}")

    if abs(moyenne - med) < 0.1 * serie.std():
        print("  -> Distribution quasi-symetrique")
    elif moyenne > med:
        print("  -> Distribution asymetrique a droite (etalee vers le haut)")
    else:
        print("  -> Distribution asymetrique a gauche (etalee vers le bas)")

    print(f"  IQR = {iqr:.4f}")
    if iqr < serie.std():
        print("  -> Donnees concentrees (IQR < ecart-type)")
    else:
        print("  -> Donnees dispersees (IQR > ecart-type)")

    print(f"  D6 - Me = {d6 - med:.4f}")
    if d6 - med > q3 - med:
        print("  -> Forte dispersion dans la partie superieure")
    else:
        print("  -> Dispersion moderee dans la partie superieure")

interpreter_distribution(df, 'Note_Stats')
interpreter_distribution(df, 'Heures_Travail')
interpreter_distribution(df, 'Age')
```

Listing 15 – Interpretation automatique

10. Diagramme de Tukey et détection des points aberrants

10.1 Principe du diagramme de Tukey

Le **diagramme de Tukey** (ou **boîte à moustaches**, *boxplot*) est une représentation graphique synthétique d'une distribution quantitative. Il repose sur les **cinq paramètres** suivants :

Symbole	Signification
Q_1	Premier quartile (25%)
Q_2	Médiane (50%)
Q_3	Troisième quartile (75%)
IQR	Écart interquartile : $IQR = Q_3 - Q_1$
L_{inf}	Borne inférieure de Tukey : $Q_1 - 1{,}5 \times IQR$
L_{sup}	Borne supérieure de Tukey : $Q_3 + 1{,}5 \times IQR$

Critère de Tukey

Un point x_i est considéré comme **aberrant** si :

$$x_i < L_{inf} \quad \text{ou} \quad x_i > L_{sup}$$

Remarque

On distingue parfois :

— **outliers modérés** : entre $1{,}5 \times IQR$ et $3 \times IQR$,

— **outliers extrêmes** : au-delà de $3 \times IQR$.

10.2 Calcul des paramètres de Tukey

```python
def parametres_tukey(serie):
    """
    Calcule les 5 parametres du diagramme de Tukey.
    Retourne un dictionnaire avec Q1, Q2, Q3, IQR, bornes et stats.
    """
    serie = serie.dropna()
    q1 = np.percentile(serie, 25)
    q2 = np.percentile(serie, 50)
    q3 = np.percentile(serie, 75)
    iqr = q3 - q1
    l_inf = q1 - 1.5 * iqr
    l_sup = q3 + 1.5 * iqr

    return {
        'Q1': q1, 'Q2': q2, 'Q3': q3,
        'IQR': iqr,
        'L_inf': l_inf, 'L_sup': l_sup,
        'min': serie.min(), 'max': serie.max(),
        'moyenne': serie.mean(), 'ecart_type': serie.std()
    }

params = parametres_tukey(df['Note_Stats'])
for k, v in params.items():
    print(f"{k:12s} = {v:.4f}")
```

Listing 16 – Calcul des paramètres de Tukey

10.3 Détection automatique des points aberrants

```python
def detecter_points_aberrants(df, colonne):
    """
    Detecte les points aberrants selon le critere de Tukey.
    Retourne un DataFrame avec les valeurs aberrantes et leur type.
    """
    serie = df[colonne].dropna()
    p = parametres_tukey(serie)

    outliers = serie[(serie < p['L_inf']) | (serie > p['L_sup'])]

    if len(outliers) == 0:
        print(f"Aucun point aberrant detecte pour '{colonne}'.")
        return pd.DataFrame(columns=['Index', 'Valeur', 'Type'])

```

```
    # Classification modere / extreme
    types = []
    for val in outliers:
        if val < p['Q1'] - 3 * p['IQR'] or val > p['Q3'] + 3 * p['IQR']:
            types.append('extreme')
        else:
            types.append('modere')

    result = pd.DataFrame({
        'Index': outliers.index,
        'Valeur': outliers.values,
        'Type': types
    })
    print(f"{len(outliers)} point(s) aberrant(s) detecte(s) pour '{colonne}' :")
    print(result)
    return result

# Application sur les variables quantitatives
for col in ['Age', 'Note_Stats', 'Heures_Travail']:
    print(f"\n--- {col} ---")
    detecter_points_aberrants(df, col)
```

Listing 17 – Fonction de détection des outliers

10.4 Tracé du diagramme de Tukey

```
def tracer_diagramme_tukey(df, colonne, couleur='skyblue',
                           seuil_extreme=3.0):
    """
    Trace un diagramme de Tukey annote avec les bornes et les outliers.
    """
    serie = df[colonne].dropna()
    p = parametres_tukey(serie)

    fig, ax = plt.subplots(figsize=(9, 6))

    # Boite a moustaches
    bp = ax.boxplot(serie, vert=True, patch_artist=True, widths=0.5,
                    boxprops=dict(facecolor=couleur, edgecolor='black'),
                    medianprops=dict(color='red', linewidth=2),
                    whiskerprops=dict(color='black', linewidth=1.5),
                    capprops=dict(color='black', linewidth=1.5),
                    flierprops=dict(marker='o', markerfacecolor='red',
                                    markersize=8, markeredgecolor='black')
                                    )

    # Bornes de Tukey
    ax.axhline(p['L_inf'], color='orange', linestyle='--', linewidth=1.5,
               label=f"L_inf = {p['L_inf']:.2f}")
    ax.axhline(p['L_sup'], color='orange', linestyle='--', linewidth=1.5,
               label=f"L_sup = {p['L_sup']:.2f}")

    # Quartiles
    for q, nom in [(p['Q1'], 'Q1'), (p['Q2'], 'Q2 (mediane)'), (p['Q3'], 'Q3')]:
        ax.axhline(q, color='green', linestyle=':', linewidth=1)
        ax.text(1.35, q, f"{nom} = {q:.2f}", va='center', fontsize=9,
                color='green')
```

```
31
32      # Moyenne
33      ax.scatter([1], [p['moyenne']], color='purple', marker='D', s=60,
34                 zorder=5, label=f"Moyenne = {p['moyenne']:.2f}")
35
36      ax.set_ylabel(colonne)
37      ax.set_title(f"Diagramme de Tukey - {colonne}")
38      ax.set_xticks([1]); ax.set_xticklabels([colonne])
39      ax.grid(axis='y', alpha=0.3)
40      ax.legend(loc='upper right', fontsize=9)
41      plt.tight_layout(); plt.show()
42
43  # Application
44  tracer_diagramme_tukey(df, 'Note_Stats')
45  tracer_diagramme_tukey(df, 'Heures_Travail', couleur='lightgreen')
```

Listing 18 – Diagramme de Tukey avec annotations

10.5 Comparaison avec une distribution normale

```
1   def detecter_outliers_zscore(serie, seuil=3):
2       """
3       Methode alternative : Z-score (suppose une distribution normale).
4       Un point est aberrant si |z| > seuil (generalement 3).
5       """
6       serie = serie.dropna()
7       z = np.abs(stats.zscore(serie))
8       outliers = serie[z > seuil]
9       print(f"Z-score (seuil={seuil}) : {len(outliers)} outlier(s)")
10      return outliers
11
12  # Comparaison Tukey vs Z-score
13  print("=== Comparaison des methodes ===\n")
14  for col in ['Note_Stats', 'Heures_Travail']:
15      print(f"--- {col} ---")
16      tukey_out = detecter_points_aberrants(df, col)
17      z_out = detecter_outliers_zscore(df[col], seuil=3)
18      print()
```

Listing 19 – Detection des outliers par la methode Z-score

Attention

La méthode du **Z-score** suppose une distribution **normale** et est sensible aux valeurs extrêmes. La méthode de **Tukey** (basée sur l'IQR) est **robuste** et ne fait aucune hypothèse sur la distribution. C'est pourquoi Tukey est préférée en analyse exploratoire.

10.6 Interprétation du diagramme de Tukey

Observation	Interprétation
Boîte large	Forte dispersion des 50% centraux
Boîte étroite	Données concentrées autour de la médiane
Médiane décalée vers Q_1	Distribution asymétrique à droite
Médiane décalée vers Q_3	Distribution asymétrique à gauche
Moustaches longues	Étendue importante
Points isolés au-delà des bornes	Valeurs aberrantes à examiner

10.7 Application au dataset

```
def analyse_tukey_complete(df):
    """
    Analyse Tukey complete pour toutes les variables quantitatives.
    """
    print("=" * 70)
    print("ANALYSE TUKEY - DETECTION DES POINTS ABERRANTS")
    print("=" * 70)

    for col in df.columns:
        if detecter_type_variable(df[col]) in [
            'quantitative_discrete', 'quantitative_continue'
        ]:
            print(f"\n{'='*70}\nVARIABLE : {col}\n{'='*70}")
            p = parametres_tukey(df[col])
            print(f"  Q1        = {p['Q1']:.3f}")
            print(f"  Q2        = {p['Q2']:.3f}")
            print(f"  Q3        = {p['Q3']:.3f}")
            print(f"  IQR       = {p['IQR']:.3f}")
            print(f"  L_inf     = {p['L_inf']:.3f}")
            print(f"  L_sup     = {p['L_sup']:.3f}")
            print(f"  Moyenne   = {p['moyenne']:.3f}")
            print(f"  Ecart-type = {p['ecart_type']:.3f}")
            detecter_points_aberrants(df, col)
            tracer_diagramme_tukey(df, col)

analyse_tukey_complete(df)
```

Listing 20 – Analyse complete de Tukey sur le dataset

11. Transformation discret → continu

```
def discret_vers_continu(df, colonne, methode='sturges'):
    serie = df[colonne].dropna()
    if methode == 'sturges':
        bornes, nb_classes = creer_classes_sturges(serie)
        nom = "Sturges"
    elif methode == 'yule':
        bornes, nb_classes = creer_classes_yule(serie)
        nom = "Yule"
    elif methode == 'fd':
```

```
        bornes, nb_classes = creer_classes_freedman_diaconis(serie)
        nom = "Freedman-Diaconis"

    df_t = df.copy()
    labels = [f"[{bornes[i]:.2f}-{bornes[i+1]:.2f}]"
              for i in range(len(bornes)-1)]
    df_t[f'{colonne}_continu'] = pd.cut(serie, bins=bornes,
                                        include_lowest=True, labels=
                                           labels)

    print(f"Methode : {nom} | Nombre de classes : {nb_classes}")
    tableau = df_t[f'{colonne}_continu'].value_counts().sort_index()
    return df_t, pd.DataFrame({
        'Classe': tableau.index,
        'Effectif': tableau.values,
        'Frequence': (tableau.values / len(serie)).round(4)
    })

df_continu, tab = discret_vers_continu(df, 'Age', 'sturges')
```

Listing 21 – Passage du discret au continu

12. Opération inverse : graphique → tableau

12.1 Reconstitution depuis un diagramme circulaire

```
def reconstituer_depuis_circulaire(nom_variable, population,
                                   categories, effectifs=None,
                                   pourcentages=None):
    if effectifs is None and pourcentages is not None:
        effectifs = [round(p * population / 100) for p in pourcentages]
    if sum(effectifs) != population:
        effectifs[-1] += population - sum(effectifs)

    frequences = [e / population for e in effectifs]
    tableau = pd.DataFrame({
        'Modalite': categories,
        'Effectif (n_i)': effectifs,
        'Frequence (f_i)': [round(f, 4) for f in frequences],
        'Frequence (%)': [round(f * 100, 2) for f in frequences],
        'Frequence cumulee': [round(fc, 4) for fc in np.cumsum(frequences)
           ]
    })
    print(f"Variable : {nom_variable} | Population : {population}")
    print("Type : Qualitative\n")
    return tableau

tab = reconstituer_depuis_circulaire('Genre', 50, ['M', 'F'],
                                     pourcentages=[50, 50])
print(tab)
```

12.2 Reconstitution depuis une courbe en escalier

```
def reconstituer_depuis_escalier(nom_variable, population,
                                 valeurs_x, frequences_cumulees):
    f_cum = np.array(frequences_cumulees)
    f_ind = np.diff(np.concatenate([[0], f_cum]))
```

```
    effectifs = [round(f * population) for f in f_ind]
    if sum(effectifs) != population:
        effectifs[-1] += population - sum(effectifs)

    type_var = ("Quantitative discrete"
                if all(isinstance(v, (int, np.integer)) for v in valeurs_x
                   )
                else "Quantitative continue (classes)")

    tableau = pd.DataFrame({
        'Valeur/Classe': valeurs_x,
        'Effectif (n_i)': effectifs,
        'Frequence (f_i)': [round(e / population, 4) for e in effectifs],
        'Frequence cumulee': [round(fc, 4) for fc in f_cum]
    })
    print(f"Variable : {nom_variable} | Population : {population}")
    print(f"Type : {type_var}\n")
    return tableau

tab = reconstituer_depuis_escalier('Age', 50,
                                   [20, 22, 24, 26, 28],
                                   [0.2, 0.4, 0.7, 0.85, 1.0])
print(tab)
```

12.3 Reconstitution depuis un histogramme

```
def reconstituer_depuis_histogramme(nom_variable, bornes_classes,
   effectifs):
    population = sum(effectifs)
    frequences = [e / population for e in effectifs]
    labels = [f"[{bornes_classes[i]:.2f} ; {bornes_classes[i+1]:.2f}]"
              for i in range(len(bornes_classes)-1)]

    tableau = pd.DataFrame({
        'Classe': labels,
        'Effectif (n_i)': effectifs,
        'Frequence (f_i)': [round(f, 4) for f in frequences],
        'Frequence cumulee': [round(fc, 4) for fc in np.cumsum(frequences)
           ]
    })
    print(f"Variable : {nom_variable} | Population : {population}")
    print("Type : Quantitative continue\n")
    return tableau

tab = reconstituer_depuis_histogramme('Note_Stats',
                                      [8, 10, 12, 14, 16, 18, 20],
                                      [5, 8, 12, 15, 7, 3])
print(tab)
```

13. Dashboard de synthèse

```
def dashboard_statistique(df):
    print("=" * 70)
    print("DASHBOARD STATISTIQUE AUTOMATIQUE")
    print("=" * 70)

```

```
    for col in df.columns:
        if col == 'ID':
            continue
        type_var = detecter_type_variable(df[col])
        print(f"\n{'='*70}\nVARIABLE : {col}\nType : {type_var}\n{'='*70}"
            )

        if 'qualitative' in type_var:
            print(tableau_qualitatif(df, col))
            if df[col].nunique() <= 6:
                tracer_diagramme_circulaire(df, col)
            else:
                tracer_tuyaux_orgues(df, col)

        elif type_var == 'quantitative_discrete':
            print(tableau_quantitatif_discret(df, col))
            tracer_diagramme_batons(df, col)
            tracer_courbe_escalier(df, col, 'discrete')
            # Parametres
            print("\n--- Parametres de position ---")
            for k, v in parametres_position(df[col]).items():
                print(f"  {k:20s} = {v:.4f}")
            print("\n--- Parametres de dispersion ---")
            for k, v in parametres_dispersion(df[col]).items():
                print(f"  {k:20s} = {v:.4f}")
            # Tukey
            print("\n--- Analyse de Tukey ---")
            detecter_points_aberrants(df, col)
            tracer_diagramme_tukey(df, col)

        elif type_var == 'quantitative_continue':
            for m in ['sturges', 'yule', 'fd']:
                print(f"--- Methode {m.upper()} ---")
                print(tableau_quantitatif_continu(df, col, m))
            tracer_histogramme(df, col, 'sturges')
            tracer_courbe_cumulative(df, col, 'sturges')
            # Quantiles
            tracer_courbe_cumulative_quantiles(df, col, 'sturges')
            # Parametres
            print("\n--- Parametres de position ---")
            for k, v in parametres_position(df[col]).items():
                print(f"  {k:20s} = {v:.4f}")
            print("\n--- Parametres de dispersion ---")
            for k, v in parametres_dispersion(df[col]).items():
                print(f"  {k:20s} = {v:.4f}")
            # Tukey
            print("\n--- Analyse de Tukey ---")
            detecter_points_aberrants(df, col)
            tracer_diagramme_tukey(df, col)

dashboard_statistique(df)
```

Listing 22 – Dashboard global avec Tukey

14. Synthèse des règles de discrétisation

Règle	Formule	Cas d'usage
Sturges	$k = 1 + 3{,}322 \log_{10}(n)$	Distributions normales, $n < 200$
Yule	$k = 2{,}5 \cdot n^{1/4}$	Distribution quelconque, $n < 1000$
Freedman–Diaconis	$h = \dfrac{2 \cdot \text{IQR}}{n^{1/3}}$	Robuste aux outliers
Tukey (détection)	$L = Q \pm 1{,}5 \times IQR$	Détection des points aberrants

15. Synthèse des paramètres statistiques

Catégorie	Paramètre	Interprétation
Position	Moyenne	Valeur centrale sensible aux extrêmes
	Médiane	Valeur centrale robuste
	Mode	Valeur la plus fréquente
Dispersion	Étendue	Amplitude totale
	IQR	Dispersion des 50% centraux (robuste)
	Écart-type	Dispersion moyenne autour de la moyenne
Quantiles	Quartiles	Division en 4 parties égales
	Déciles	Division en 10 parties égales
Forme	Asymétrie	Symétrie de la distribution
	Aplatissement	Concentration autour de la moyenne

```python
def synthese_statistique(df, colonne):
    """Synthese complete des parametres statistiques."""
    serie = df[colonne].dropna()

    print("=" * 70)
    print(f"SYNTHESE STATISTIQUE - {colonne}")
    print("=" * 70)

    # Position
    print("\n[POSITION]")
    print(f"  Moyenne          = {serie.mean():.4f}")
    print(f"  Mediane          = {serie.median():.4f}")
    print(f"  Mode             = {serie.mode().iloc[0]:.4f}")

    # Quantiles
    print("\n[QUANTILES]")
    for p, nom in [(25, 'Q1'), (50, 'Q2 (Me)'), (60, 'D6'), (75, 'Q3')]:
        print(f"  {nom:12s}   = {np.percentile(serie, p):.4f}")
```

```
19
20      # Dispersion
21      print("\n[DISPERSION]")
22      print(f"  Etendue          = {serie.max() - serie.min():.4f}")
23      print(f"  IQR (Q3-Q1)      = {np.percentile(serie,75) - np.percentile(
            serie,25):.4f}")
24      print(f"  Variance         = {serie.var(ddof=1):.4f}")
25      print(f"  Ecart-type       = {serie.std(ddof=1):.4f}")
26      print(f"  CV (%)           = {serie.std(ddof=1)/serie.mean()*100:.2f}")
27
28      # Forme
29      print("\n[FORME]")
30      print(f"  Asymetrie        = {serie.skew():.4f}")
31      print(f"  Aplatissement    = {serie.kurtosis():.4f}")
32
33      # Extremes
34      print("\n[EXTREMES]")
35      print(f"  Minimum          = {serie.min():.4f}")
36      print(f"  Maximum          = {serie.max():.4f}")
37
38  synthese_statistique(df, 'Note_Stats')
```

Listing 23 – Fonction de synthese globale

16. Exercices supplémentaires

1. **Application** : Chargez un dataset réel (`titanic.csv`, `iris.csv`) et appliquez le dashboard complet.
2. **Comparaison** : Pour une même variable continue, comparez les histogrammes obtenus avec les 3 règles. Quelle méthode donne la meilleure visualisation ?
3. **Points aberrants** :
   (a) Générez une série de 100 valeurs suivant une loi normale $N(0,1)$ et injectez-y manuellement 3 outliers (-8, 10, 15).
   (b) Détectez-les avec la méthode de Tukey puis avec le Z-score.
   (c) Comparez les résultats et concluez sur la robustesse des deux méthodes.
4. **Interpolation** : Construisez un histogramme d'une variable continue, puis déterminez graphiquement Q_1, Me, Q_3 par interpolation. Comparez avec les valeurs exactes calculées sur les données brutes.
5. **Courbe cumulative** : À partir de la courbe cumulative, déterminez D_1, D_3, D_6, D_9. Interprétez chaque décile.
6. **Opération inverse avancée** : À partir d'un diagramme circulaire fourni en image, estimez les pourcentages par analyse d'image (OpenCV) et reconstituez le tableau.
7. **Dashboard interactif** : Utilisez Plotly ou Dash pour créer un dashboard interactif où l'utilisateur sélectionne la variable et la méthode de discrétisation.
8. **Tukey par groupe** : Construisez un diagramme de Tukey **par groupe** (ex. : `Note_Stats` selon `Genre`) en utilisant `seaborn.boxplot()`. Interprétez les différences entre groupes.

Bibliographie

— Formule de Sturges : $k = 1 + 3{,}322 \log_{10}(n)$
— Règle de Yule : $k = 2{,}5 \cdot n^{1/4}$
— Règle de Freedman–Diaconis : $h = 2 \cdot \mathrm{IQR}/n^{1/3}$

- **Diagramme de Tukey** : J. W. Tukey, *Exploratory Data Analysis*, Addison-Wesley, 1977.
- Détection des outliers : critère $1{,}5 \times IQR$ (Tukey) et Z-score.
- Paramètres de position et de dispersion : moyenne, médiane, mode, quartiles, déciles, variance, écart-type, CV.
- Représentations graphiques : tuyaux d'orgues, diagramme circulaire, diagramme en bâtons, courbe en escalier, histogramme, courbe cumulative, boîte à moustaches.
- Documentation Pandas, NumPy, Matplotlib, Seaborn et SciPy.